Chapter 2: Application Layer

Part 1: Principles and Socket Programming

Roadmap

- Where is app layer implemented?
- Client-server vs peer-to-peer architectures
- Process addressing
- TCP, UDP overview
- Socket programming

Some network apps

- social networking
- Web
- text messaging
- e-mail
- multi-user network games
- streaming stored video (YouTube, Hulu, Netflix)
- P2P file sharing
- voice over IP (e.g., Skype)
- real-time video conferencing (e.g., Zoom)
- Internet search
- remote login
- …

Q: your favorites?

Preliminaries

- IP address: network *identifier* of a host

- IP assignment can be:
  - **dynamic:** IP address of host may change over time
  - **permanent:** IP address of host is "always" the same

- IP is part of *Network Layer* and will be further discussed in Chapters 4-5.

Application Layer (we are here)
Transport Layer
Network Layer
Data Link Layer
Physical Layer

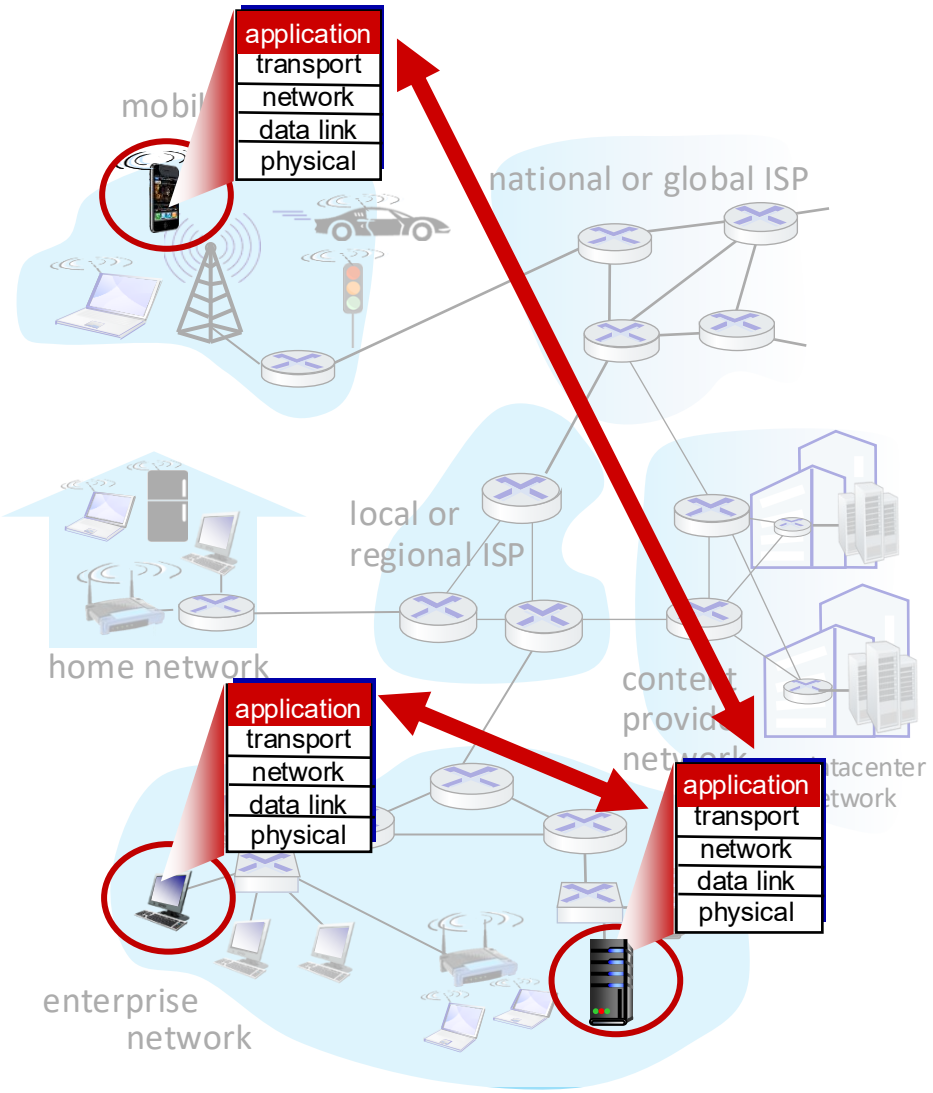
Creating a network app

write programs that:

- run on (different) end systems
- communicate over network
- e.g., web server software communicates with browser software

no need to write software for network-core devices

- network-core devices do not run user applications
- applications on end systems allows for rapid app development, propagation



Client-server paradigm

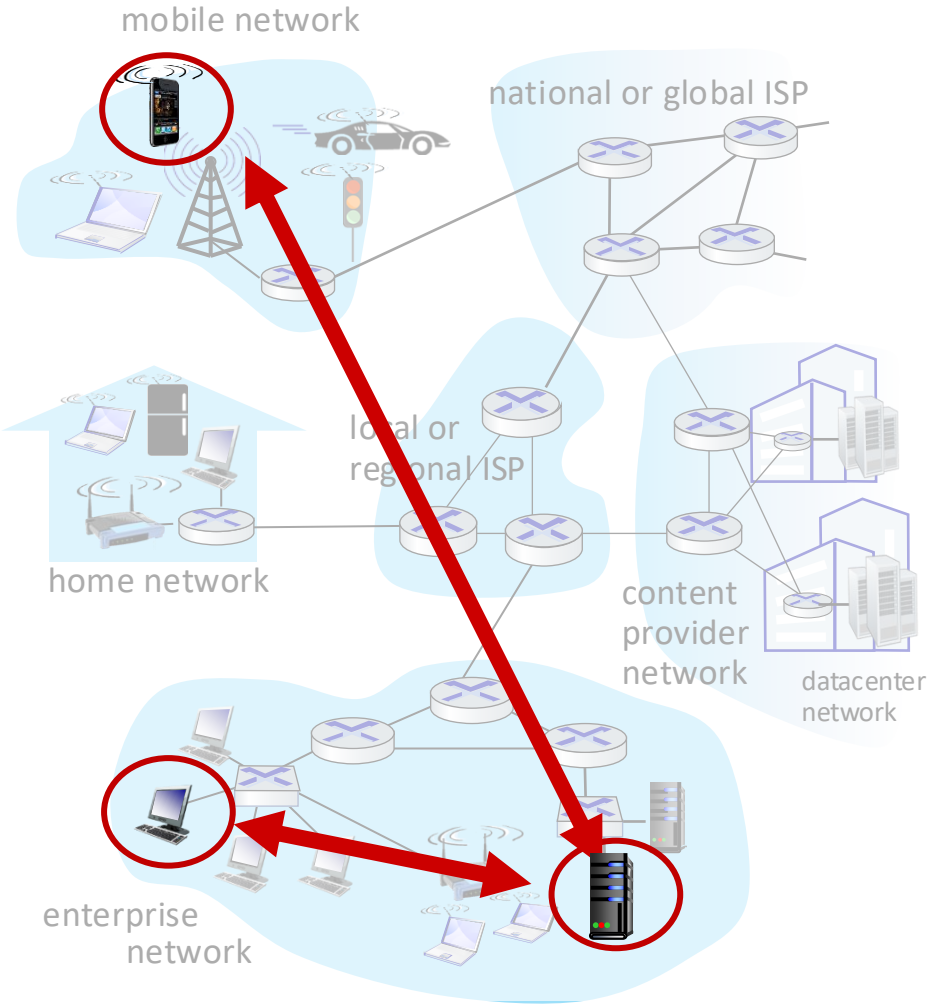
server:

- always-on host
- permanent IP address
- often in data centers, for scaling

clients:

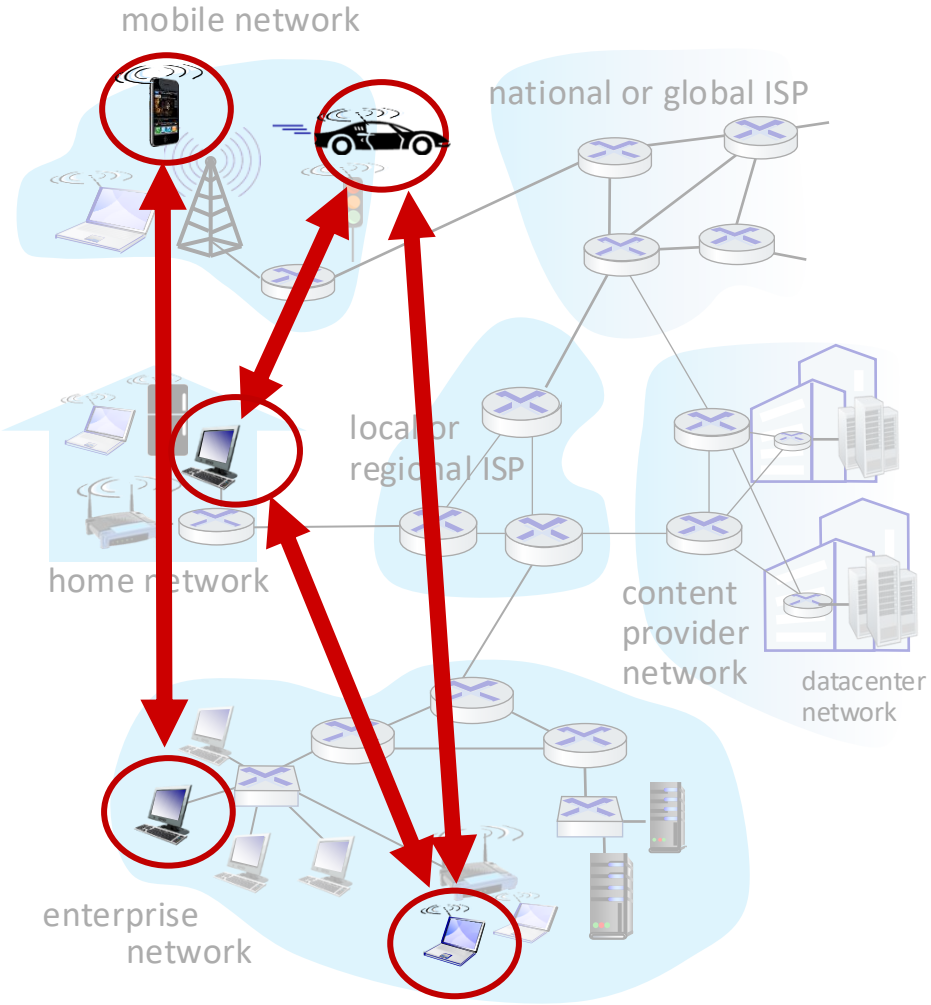
- contact, communicate with server
- may be intermittently connected
- may have dynamic IP addresses
- do *not* communicate directly with each other

- examples: HTTP, IMAP, FTP



Peer-to-peer architecture

- *no* always-on server
- arbitrary end systems directly communicate
- peers request service from other peers, provide service in return to other peers
  - *self scalability* – new peers bring new service capacity, as well as new service demands
- peers are intermittently connected and change IP addresses
  - complex management
- example: P2P file sharing [BitTorrent]



Client-server vs Peer-to-peer

- Client-server
- Peer-to-peer

For details on P2P, Chapter 2.5.

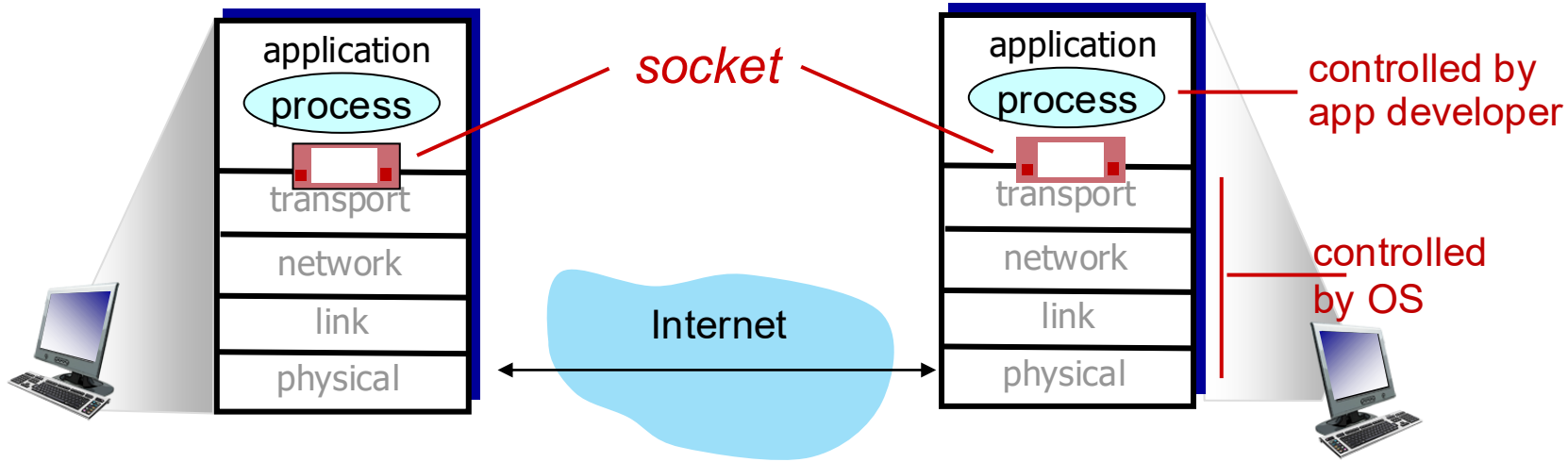
Definitions

- A ***process*** is a program running within a host:
  - *client* is the process that initiates communication
  - *server* is the process that waits to be contacted
    (permanent IP address, always on)

- Side note: in P2P, every host runs client & server processes

Sockets: communicating with lower layers

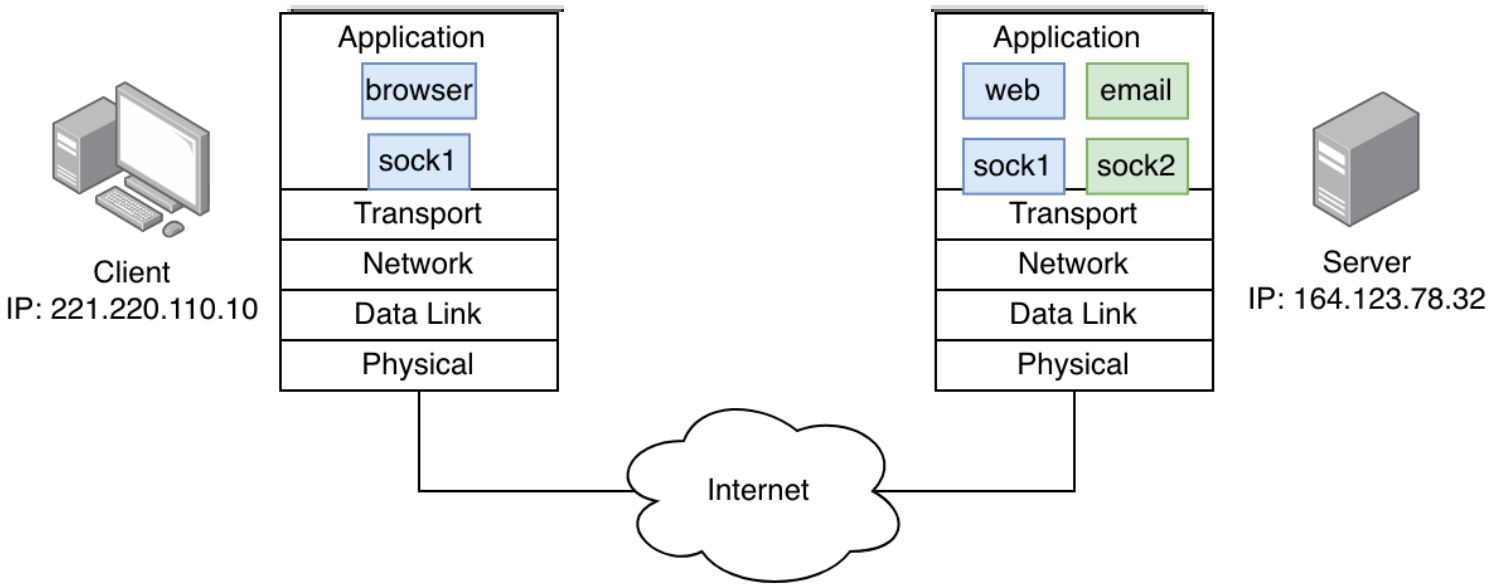
- Process sends/receives *messages* to/from *sockets*
- Socket analogous to mailing system
  - sending process "drops" message: collection boxes, mail pickup, USPS office
  - sending process relies on transport infrastructure to deliver message to receiving process' mailbox
  - two sockets involved: one on each side



Addressing processes

IP identifies hosts, but:

- server host may host multiple server processes
  e.g., *web* server and *email* server
- ***how does a client know which process to reach out?***



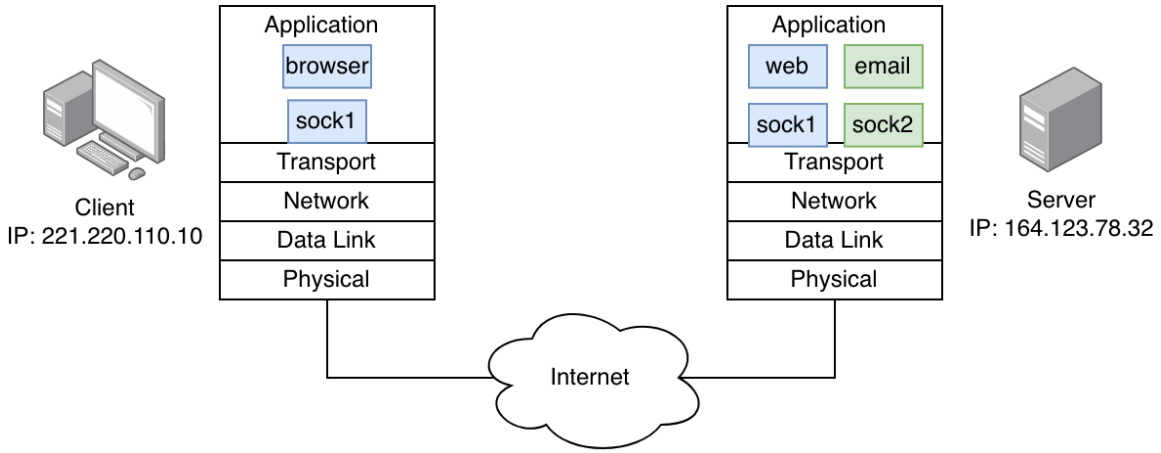
Addressing processes

- Process *identifier* includes
  - IP address
  - **port number** (we will discuss them in detail in Chapter 3)

- Example of well-known port numbers:
  - HTTP server: 80
  - SSH server: 22
  - mail server: 25
  - DNS server: 53
- e.g., to send HTTP message to *gaia.cs.umass.edu* web server:
  - IP address 128.119.245.12 and port number 80

Transport layer protocols

- Two popular transport layer protocols: **TCP** and **UDP**
- Each uses slightly different socket interfaces
- App developer needs to specify which to use



TCP vs UDP

Note: TCP and UDP will be further discussed in Chapter 3, Transport Layer.

TCP service:

- *reliable transport* between sending and receiving processes
- *flow control*: sender won't overwhelm receiver
- *congestion control:* throttle sender when network is overloaded
- *connection-oriented:* setup required between sender and receiver processes

UDP service:

- *unreliable data transfer*: no guarantee message will be delivered to receiving process (best effort)
- *does not provide:* flow control, congestion control
- *connectionless:* no connection setup
- *simpler* and often faster than TCP (less overhead)

Socket programming

- All popular programming languages provide socket libraries
- Socket programming involves identifying and understanding them
- Important considerations when developing a network app:
  - **server:**
    - "publish" IP and port number
    - define appropriate transport protocol
  - **client:**
    - identify IP and port number of server process
    - identify transport layer in use

Socket programming with UDP

server (running on serverIP)

create socket, port= x:
serverSocket =
socket(AF_INET,SOCK_DGRAM)

read datagram from
serverSocket

write reply to
serverSocket
specifying
client address,
port number

client

create socket:
clientSocket =
socket(AF_INET,SOCK_DGRAM)

Create datagram with serverIP address
And port=x; send datagram via
clientSocket

read datagram from
clientSocket

close
clientSocket

Toy example: Lower to Upper case

- Client reads a line from keyboard and sends it to server
- Server receives data and converts to uppercase
- Server sends uppercase message to client
- Client displays received message

Note: Examples assume Python 3

UDP client

Python UDPClient

include Python's socket library →
```
from socket import *
```
```
serverName = 'hostname'
serverPort = 12000
```
create UDP socket →
```
clientSocket = socket(AF_INET,
                      SOCK_DGRAM)
```
get user keyboard input →
```
message = input('Input lowercase sentence:')
```
attach server name, port to message; send into socket →
```
clientSocket.sendto(message.encode(),
                    (serverName, serverPort))
```
read reply data (bytes) from socket →
```
modifiedMessage, serverAddress =
                    clientSocket.recvfrom(2048)
```
print out received string and close socket →
```
print(modifiedMessage.decode())
clientSocket.close()
```

UDP server

Python UDPServer

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_DGRAM)
serverSocket.bind(('', serverPort))
print('The server is ready to receive')
while True:
    message, clientAddress = serverSocket.recvfrom(2048)
    # e.g.,
    modifiedMessage = message.decode().upper()
    serverSocket.sendto(modifiedMessage.encode(),
                        clientAddress)
```

create UDP socket →

bind socket to local port number 12000 →

loop forever →

Read from UDP socket into message, getting client's address (client IP and port) →

send upper case string back to this client →

Socket programming with TCP

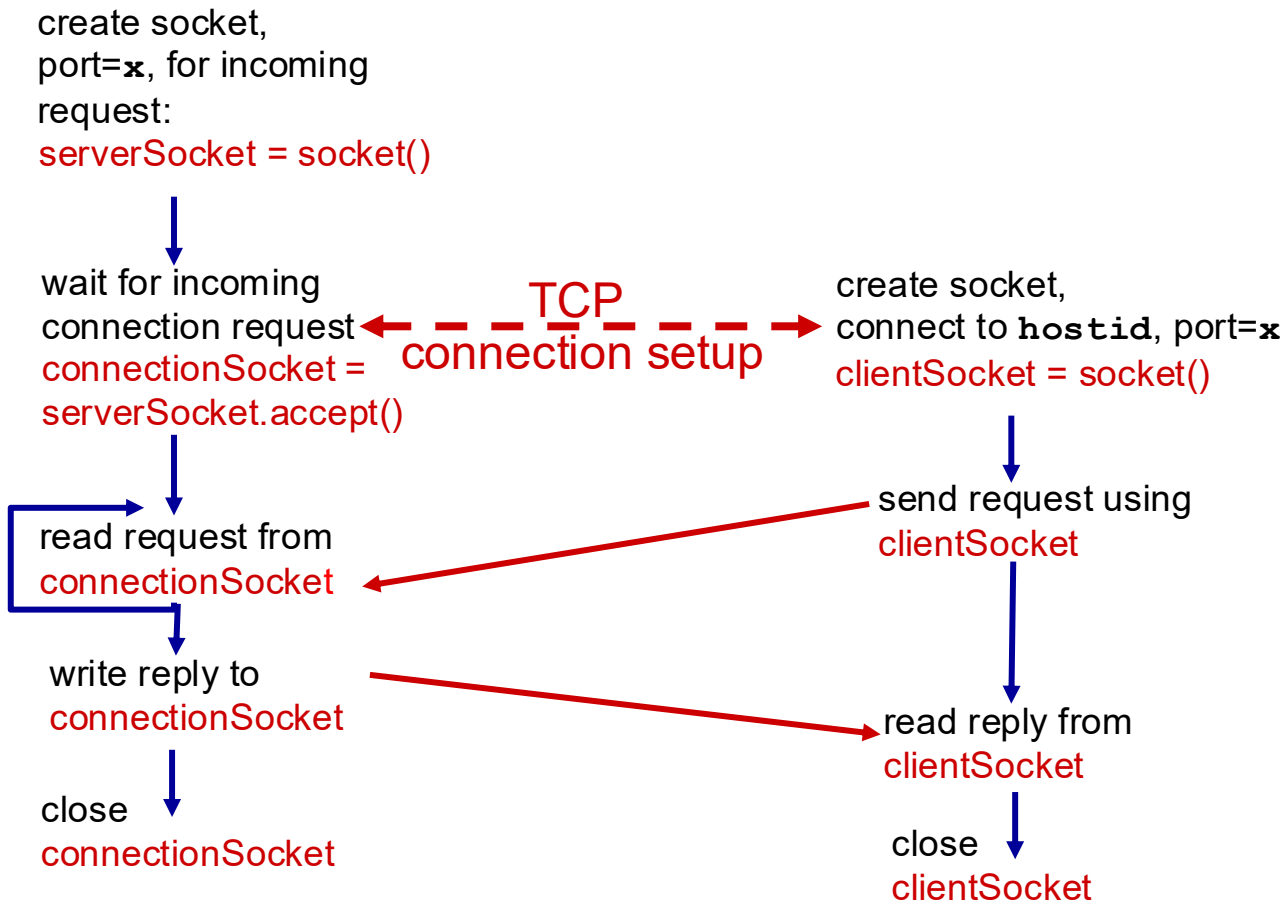
To provide reliable data transfer and flow, congestion control, TCP requires connection setup:

- first, client-server processes establish connection before communication
- then, client-server communicate over *exclusive* socket connection
  - server listens to connection requests using *common* socket
  - server creates new *private* socket to each client

Socket programming with TCP

server (running on hostid)

client 



TCP client

Python TCPClient

```
from socket import *
serverName = 'servername'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName,serverPort))
sentence = input('Input lowercase sentence:')
clientSocket.send(sentence.encode())
modifiedSentence = clientSocket.recv(1024)
print ('From Server:', modifiedSentence.decode())
clientSocket.close()
```

create TCP socket for server, remote port 12000

No need to attach server name, port

Note: this code update (2023) to Python 3

TCP server

Python TCPServer

```
 from socket import *
serverPort = 12000
serverSocket = socket(AF_INET,SOCK_STREAM)
serverSocket.bind(('',serverPort))
serverSocket.listen(1)
print('The server is ready to receive')
while True:
    connectionSocket, addr = serverSocket.accept()

    sentence = connectionSocket.recv(1024).decode()
    capitalizedSentence = sentence.upper()
    connectionSocket.send(capitalizedSentence.
                                        encode())
    connectionSocket.close()
```

create TCP welcoming socket →

server begins listening for incoming TCP requests →

loop forever →

server waits on accept() for incoming requests, new socket created on return →

read bytes from socket (but not address as in UDP) →

close connection to this client (but *not* welcoming socket) →